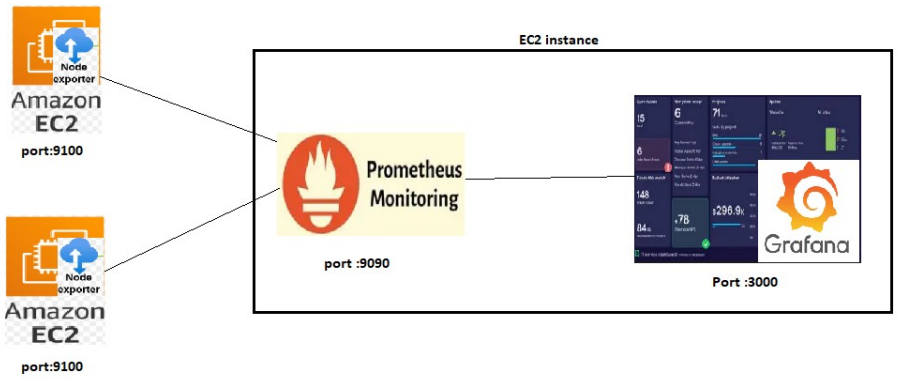
Mukesh Gouda 20/8/2026

Monitoring System

Task - Create a Monitoring System to track the server health using node_exporter , Prometheous and grafana.Also integrate alerting system (alert manager + Pagerduty).aslo ensure it support ASG



Requirements –

1. **3 ec2 instances[ec2-1 (Prometheous +Grafan) ,ec2-2 node_exporter installed]**
2. **node_exporter –** its an agent collects the data from the system
3. **Prometheous –** its scrapes (fetch and labels) metrics from the node_exporter
4. **Grafana** - its visualize the data
5. **SG configuration (Monitoring – 22,9090,3000,9093 , node_exporter -22,9100)**
6. **Install alert manager +integrate pagerduty**

Repo for reference–

itmukesh-16/monitoring-alerting-pagerdutyincident

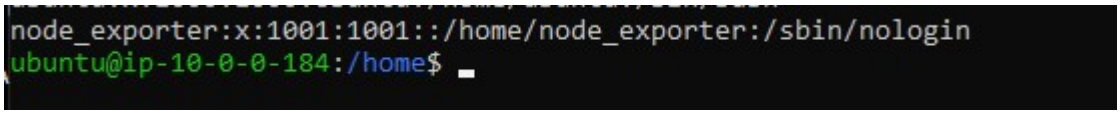
Mukesh Gouda 20/8/2026

Implementation:

Application/node_server configutation:

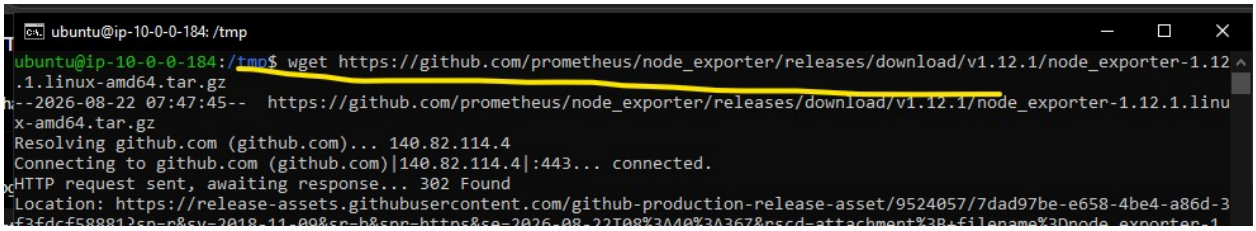
1. Create two servers .
2. Update package manager
   sudo apt update
   sudo apt upgrade -y

3. Login as non root user -> create non root user ->
   sudo useradd --no-create-home --shell /sbin/nologin node_exporter



   verify - id node_exporter

4. Download Node Exporter
   cd /tmp
   wget https://github.com/prometheus/node_exporter/releases/download/v1.12.1/node_exporter-1.12.1.linux-amd64.tar.gz



5. Extract:
   tar -xzf node_exporter-1.12.1.linux-amd64.tar.gz

```
ubuntu@ip-10-0-0-184:/tmp$ tar -xzf node_exporter-1.12.1.linux-amd64.tar.gz
ubuntu@ip-10-0-0-184:/tmp$ ls
node_exporter-1.12.1.linux-amd64
node_exporter-1.12.1.linux-amd64.tar.gz
snap-private-tmp
systemd-private-38edf63723564bdc980c524563882478-ModemManager.service-glepNB
systemd-private-38edf63723564bdc980c524563882478-chrony.service-c9FpHd
systemd-private-38edf63723564bdc980c524563882478-irqbalance.service-En5jpb
systemd-private-38edf63723564bdc980c524563882478-polkit.service-gyJ2UX
systemd-private-38edf63723564bdc980c524563882478-systemd-logind.service-sAKBJ8
ubuntu@ip-10-0-0-184:/tmp$
```

6. Now move the node_exporter file to bin folder and change ownership

   # sudo mv /tmp/node_exporter-1.12.1.linux-amd64/node_exporter /usr/local/bin/

   #sudo chown node_exporter:node_exporter /usr/local/bin/node_exporter

7. Create a node_exporter service to autostart on server reboot.

   sudo nano /etc/systemd/system/node_exporter.service

   ```
   [Unit]
   Description=Node Exporter
   After=network.target

   [Service]
   User=node_exporter
   Group=node_exporter
   ExecStart=/usr/local/bin/node_exporter
   Restart=always

   [Install]
   WantedBy=multi-user.target
   ```

   Save and exit [CTRL +O → enter →CTRL +X]

   sudo systemctl daemon-reload
   sudo systemctl enable --now node_exporter
   sudo systemctl status node_exporter

```
ubuntu@ip-10-0-0-184:/tmp$ sudo systemctl status node_exporter
● node_exporter.service - Node Exporter
     Loaded: loaded (/etc/systemd/system/node_exporter.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-08-22 08:06:32 UTC; 6s ago
 Invocation: c8a2566b82534a48893c13ab4e037082
   Main PID: 2218 (node_exporter)
      Tasks: 5 (limit: 627)
     Memory: 2.4M (peak: 2.6M)
        CPU: 17ms
     CGroup: /system.slice/node_exporter.service
             └─2218 /usr/local/bin/node_exporter
```

Verify non user - systemctl show node_exporter -p User -p Group
Verify node_exporter –

Run below in same server
curl http://localhost:9100/metrics
or
public ip of node-server:9100

expected o/p
node_cpu_seconds_total
node_memory_MemAvailable_bytes
node_filesystem_avail_bytes

EC2-1: Prometheus configuration:

8. Login to prometheous +grafana server .(since we have configured both in same)
   cd /tmp

   wget
   https://github.com/prometheus/prometheus/releases/download/v3.5.0/prometheus-3.5.0.linux-amd64.tar.gz

   tar -xzf prometheus-3.5.0.linux-amd64.tar.gz

   sudo mv prometheus-3.5.0.linux-amd64 /opt/Prometheus

   /opt/prometheus/prometheus –version

```
ubuntu@ip-10-0-0-184:/opt/prometheus$ ls
LICENSE  NOTICE  prometheus  prometheus.yml  promtool
ubuntu@ip-10-0-0-184:/opt/prometheus$
```

9. Create Prometheus non root user

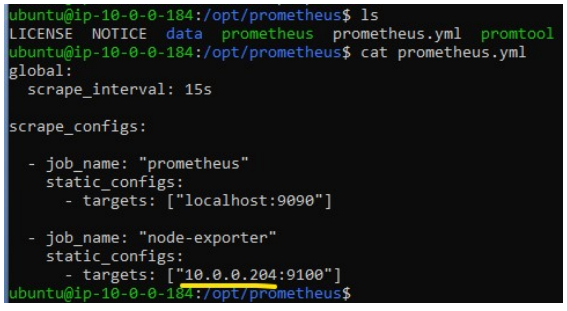
   sudo useradd --no-create-home --shell /sbin/nologin Prometheus
   sudo mkdir -p /opt/prometheus/data
   sudo chown -R prometheus:prometheus /opt/Prometheus

10. Prometheous Yaml configuration

Static Metric collection from fixed pvt ip server

sudo nano /opt/prometheus/prometheus.yml

```
global:
  scrape_interval: 15s

scrape_configs:

 - job_name: "prometheus"
   static_configs:
     - targets: ["localhost:9090"]

 - job_name: "node-exporter"
   static_configs:
     - targets: ["<EC2-2-PRIVATE-IP> the server you want to monitor:9100"]
```



Dynamic Metric collection

Here we have manually added the node-server private ip to the yaml file.
But in dynamic situation like ASG it not possible to add manually and also remove once the server deleted .

To overcome this will need to implement the **EC2 service discovery** to auto identify the the server having node agent and tell the prometheous to scrape the data.
New EC2 launch → Node Exporter install → add tag (Monitoring=true) → Prometheus discovers it.
Mandatory :
Prometheus EC2 IAM role → ec2:DescribeInstances
Node Exporter SG → TCP 9100 allowed from Prometheus SG and add the tag

11. Create a prometheous service

sudo nano /etc/systemd/system/prometheus.service

```
[Unit]
Description=Prometheus
After=network.target

[Service]
User=prometheus
Group=prometheus
ExecStart=/opt/prometheus/prometheus \
  --config.file=/opt/prometheus/prometheus.yml \
  --storage.tsdb.path=/opt/prometheus/data

Restart=always
[Install]
WantedBy=multi-user.target
```

12. Check configuration

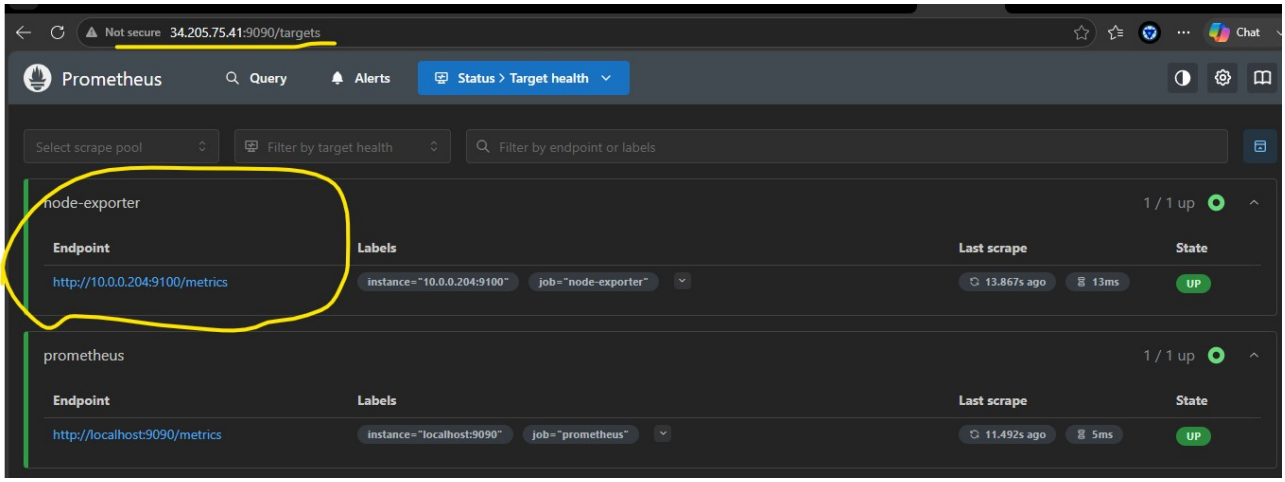
/opt/prometheus/promtool check config /opt/prometheus/prometheus.yml

```
ubuntu@ip-10-0-0-184:/opt/prometheus$ /opt/prometheus/promtool check config /opt/prometheus/prometheus.yml
Checking /opt/prometheus/prometheus.yml
 SUCCESS: /opt/prometheus/prometheus.yml is valid prometheus config file syntax
```

sudo systemctl daemon-reload
sudo systemctl enable --now Prometheus
sudo systemctl status prometheus

13. test connectivity –

    on Prometheus server run
    curl http://<EC2-2-PRIVATE-IP > the server you want to monitor :9100/metrics
    or



Configure Grafana:

14. Create grafana repo

    sudo tee /etc/yum.repos.d/grafana.repo <<'EOF'
    [grafana]
    name=grafana
    baseurl=https://rpm.grafana.com
    repo_gpgcheck=1
    enabled=1
    gpgcheck=1
    gpgkey=https://rpm.grafana.com/gpg.key
    EOF

    Or
    For ubantu
    Add the repository -

echo "deb [signed-by=/etc/apt/keyrings/grafana.gpg] https://apt.grafana.com stable main" | sudo tee /etc/apt/sources.list.d/grafana.list

Package list update in APT -
sudo apt update

installing grafana –
sudo apt install grafana –y

sudo systemctl enable --now grafana-server
sudo systemctl status grafana-server

15. Now open and test installation in browser

http:<EC2 public ip >:3000

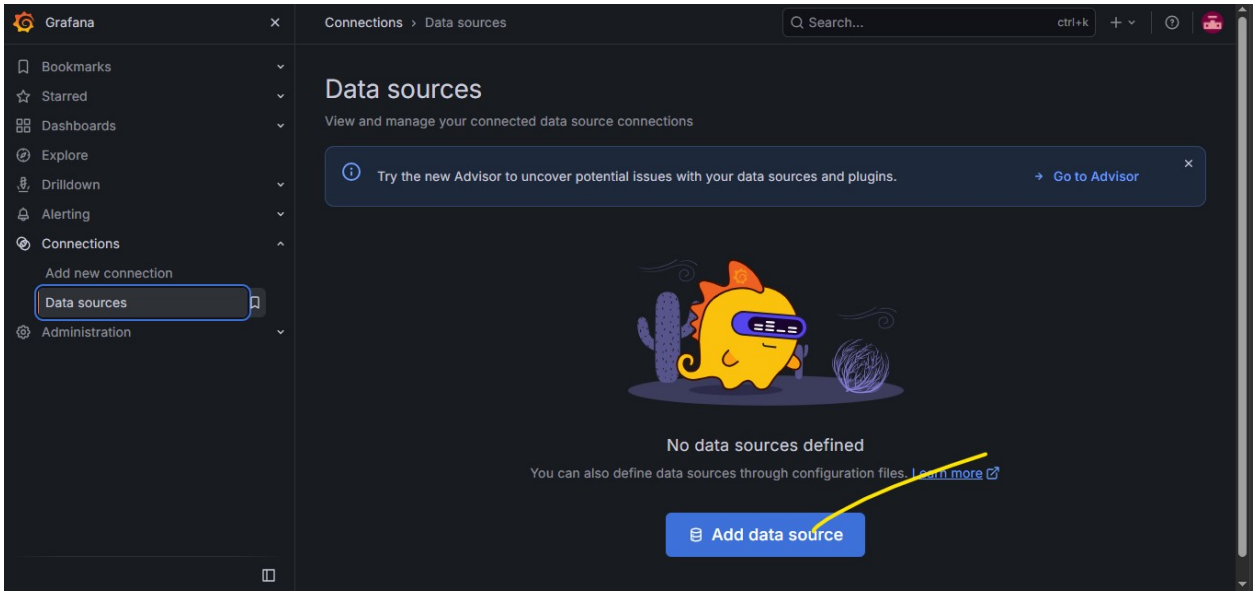
default user :admin
default password :admin

you can change the admin and password

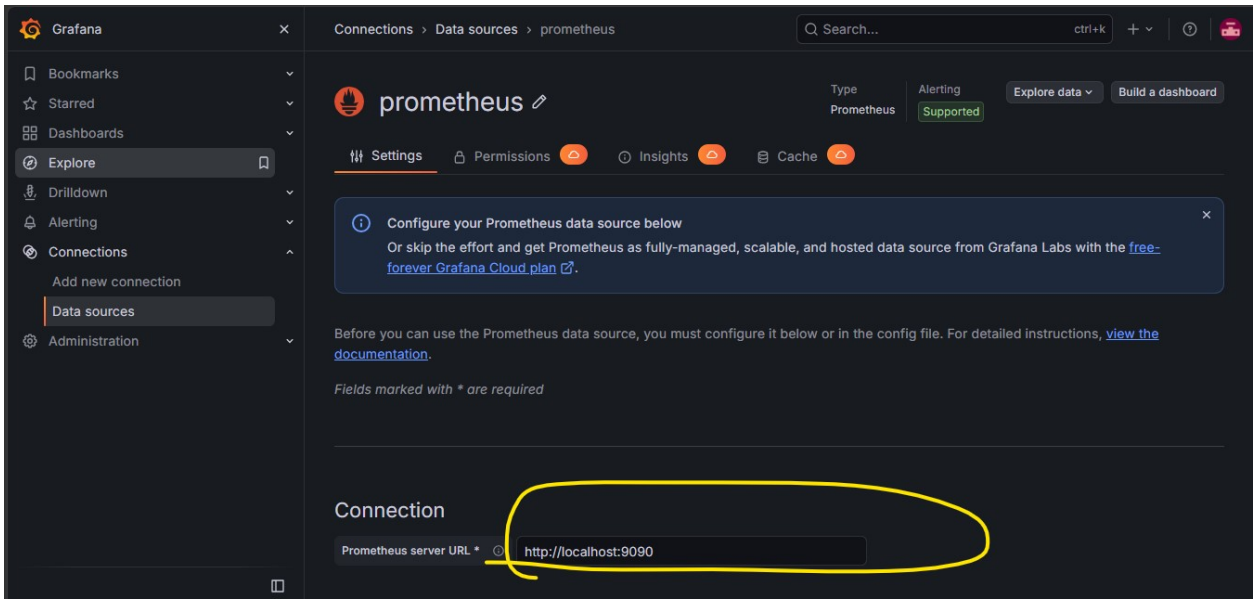
Now configure Prometheus -> grafana :

Add connection > prometheus

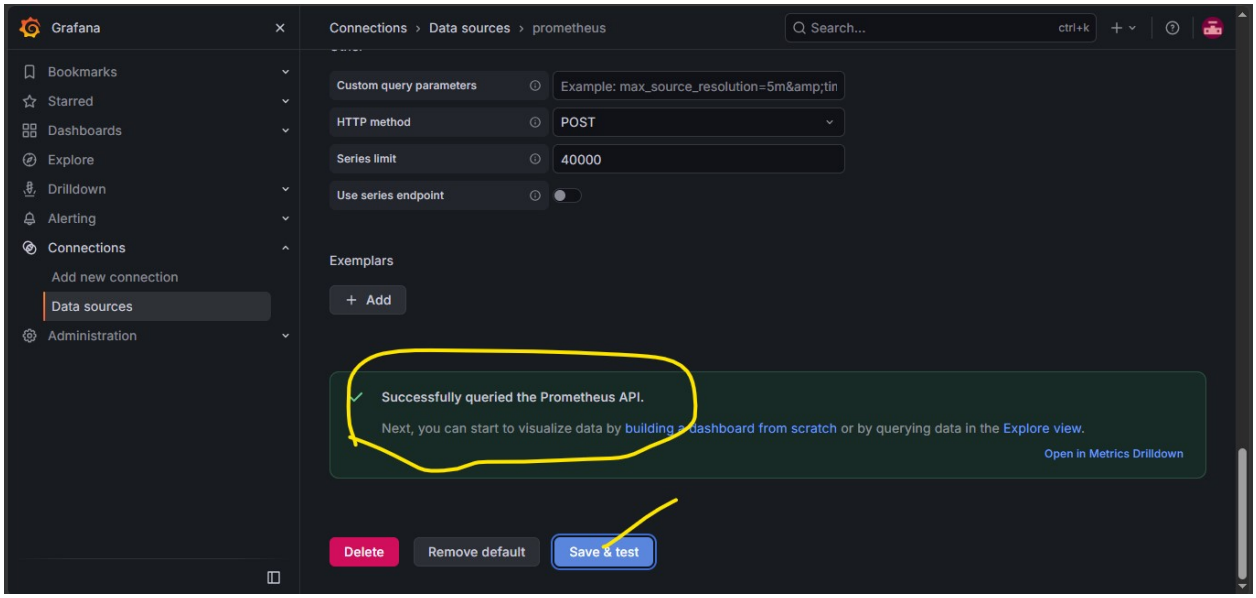
Mukesh Gouda 20/8/2026



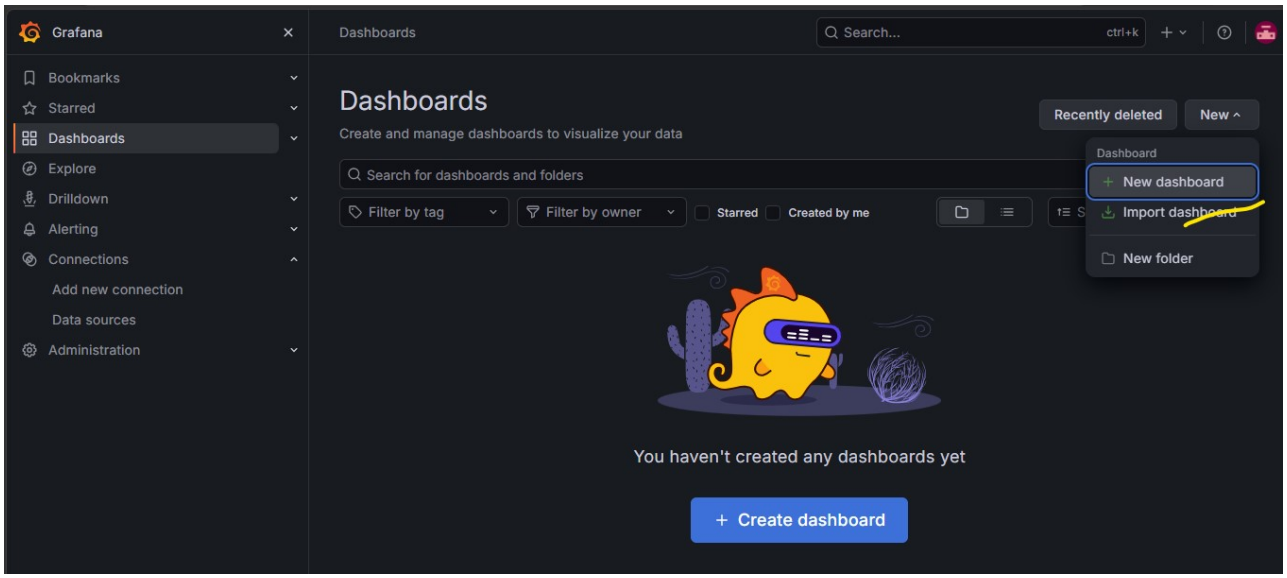
Give source ie Prometheus running server private ip (since both Prometheus and grafana running on the same server we have given localhost or 127.0.0.1)



Mukesh Gouda 20/8/2026

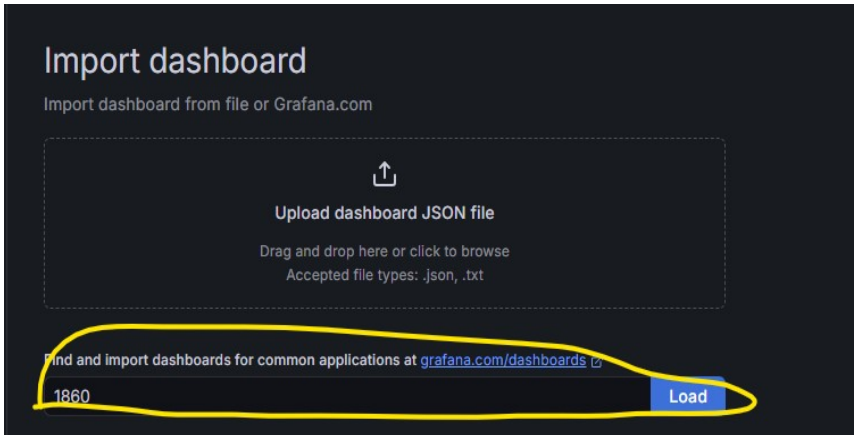


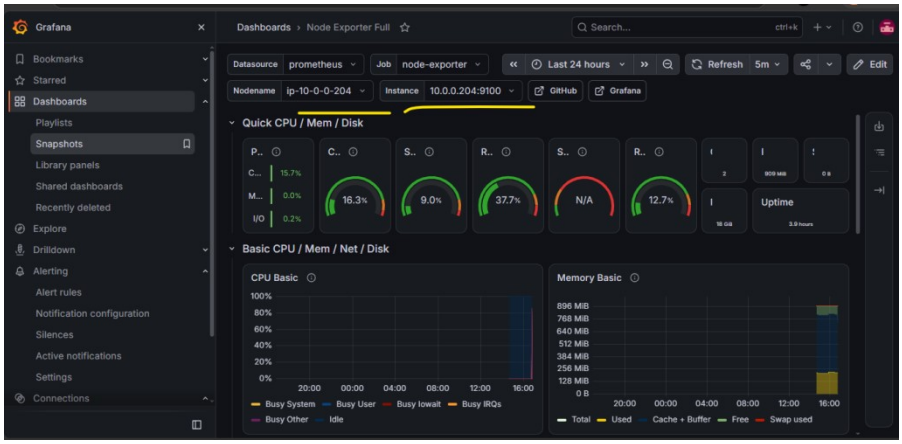
Now import dashboard



Mukesh Gouda 20/8/2026

Give dashboard template code >laod >import





Now you can use stress in node_exporter to validate if the Prometheus fetching the log and grafana showing in the dashboard or not

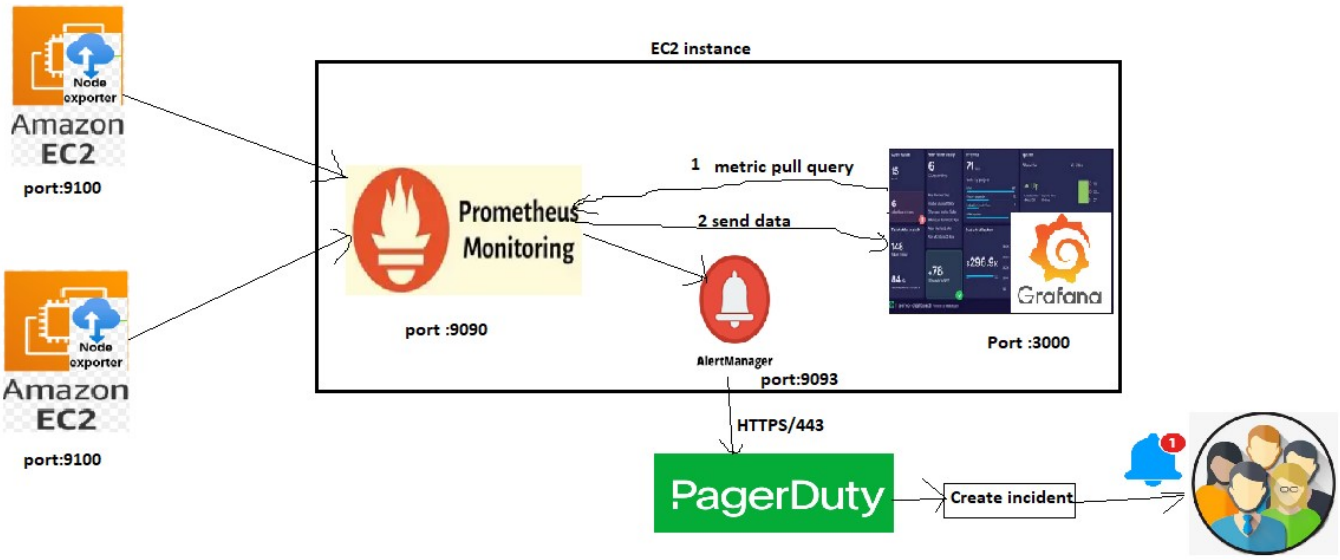
sudo apt install stress -y

stress -c 4 --timeout 300

Same way we can install node_exporter in another server and update the private ip in the **prometheous.yml** file (**/opt/prometheus/prometheus.yml**) .

Alert manager configuration +pagerduty integration

Mukesh Gouda 20/8/2026



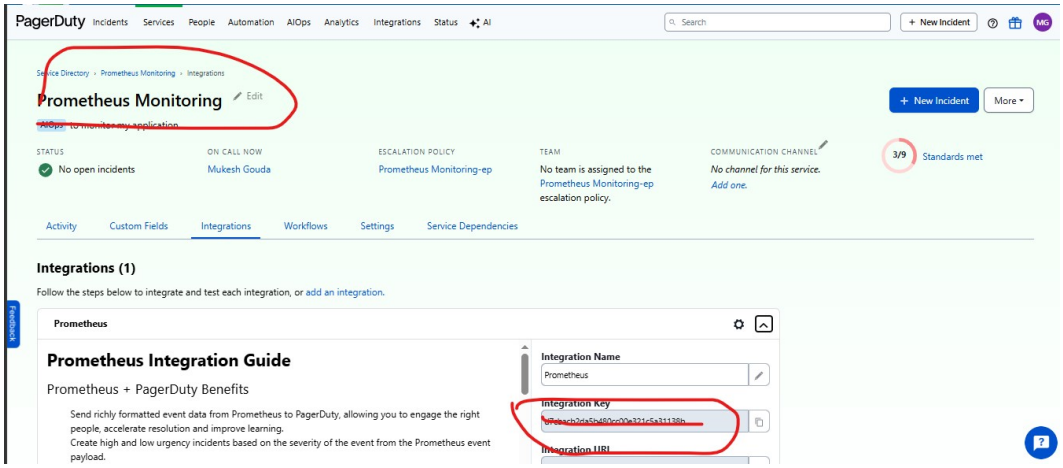
Pagerduty – based on the configuration it will trigger an incident.

Alert manager – it will collect metrics from Prometheus and sent to pagerduty

Login to pagerduty

Services -> Service Directory -> New Service

Create service (Prometheus monitoring) and store the integration key



Mukesh Gouda 20/8/2026

Configure alert manager in Prometheus + grafana running server(ec2-1):

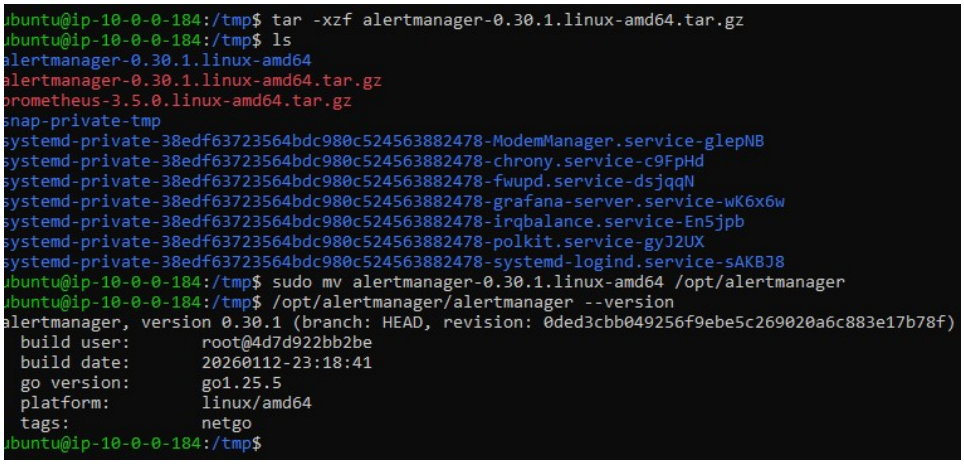
cd /tmp

wget https://github.com/prometheus/alertmanager/releases/download/v0.30.1/alertmanager-0.30.1.linux-amd64.tar.gz

tar -xzf alertmanager-0.30.1.linux-amd64.tar.gz

sudo mv alertmanager-0.30.1.linux-amd64 /opt/alertmanager

/opt/alertmanager/alertmanager –version



Create a non user for alert manager:

sudo useradd --no-create-home --shell /sbin/nologin alertmanager

sudo mkdir -p /opt/alertmanager/data

sudo chown -R alertmanager:alertmanager /opt/alertmanager

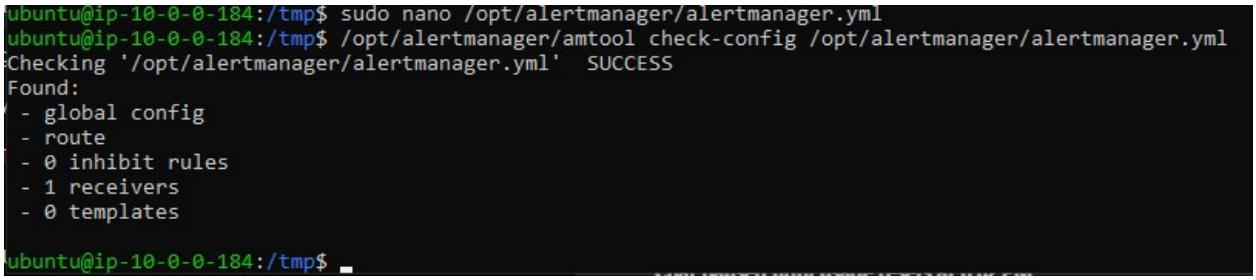
Alertmanager Configuration:

sudo nano /opt/alertmanager/alertmanager.yml

```
global:
  resolve_timeout: 5m
```

Mukesh Gouda 20/8/2026

```
route:
  receiver: pagerduty
receivers:
  - name: pagerduty
    pagerduty_configs:
      - routing_key: "YOUR_PAGERDUTY_INTEGRATION_KEY"
        send_resolved: true
```



Create a service for alert manager

sudo nano /etc/systemd/system/alertmanager.service

```
[Unit]
Description=Alertmanager
After=network.target
[Service]
User=alertmanager
Group=alertmanager
ExecStart=/opt/alertmanager/alertmanager \
```

Mukesh Gouda 20/8/2026

```
 --config.file=/opt/alertmanager/alertmanager.yml \
 --storage.path=/opt/alertmanager/data
Restart=always
[Install]
WantedBy=multi-user.target
```

sudo systemctl daemon-reload

sudo systemctl enable --now alertmanager

sudo systemctl status alertmanager

Test alert manager

curl http://localhost:9093/-/healthy

```
ubuntu@ip-10-0-0-184:/tmp$ curl http://localhost:9093/-/healthy
OKubuntu@ip-10-0-0-184:/tmp$
```

or

http://<EC2-1-PUBLIC-IP>:9093

Now create a file named rules to store the alert rules

sudo mkdir -p /opt/prometheus/rules

sudo nano /opt/prometheus/rules/node_alerts.yml

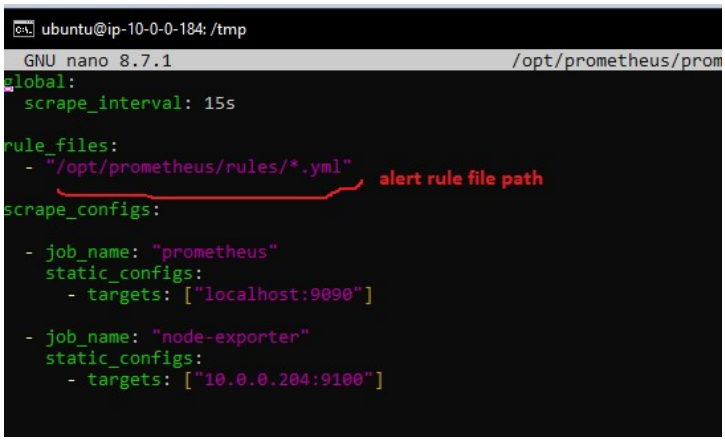
```
groups:
 - name: node_alerts
   rules:
    - alert: HighCPUUsage
      expr: 100 - (avg by(instance) (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
      for: 2m
```

```
    labels:
      severity: critical
    annotations:
      summary: "High CPU usage detected"
      description: "CPU usage on {{ $labels.instance }} is above 80% for more than 2 minutes."
```

Validate

/opt/prometheus/promtool check config /opt/prometheus/prometheus.yml

Prometheus Config Update

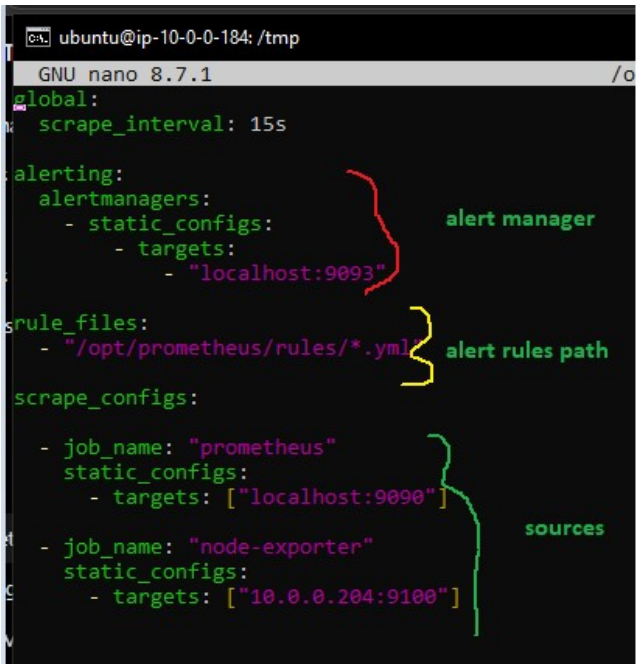


Tell Prometheus About Alertmanager:

sudo nano /opt/prometheus/prometheus.yml

```
global:
  scrape_interval: 15s


alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - "localhost:9093"
```

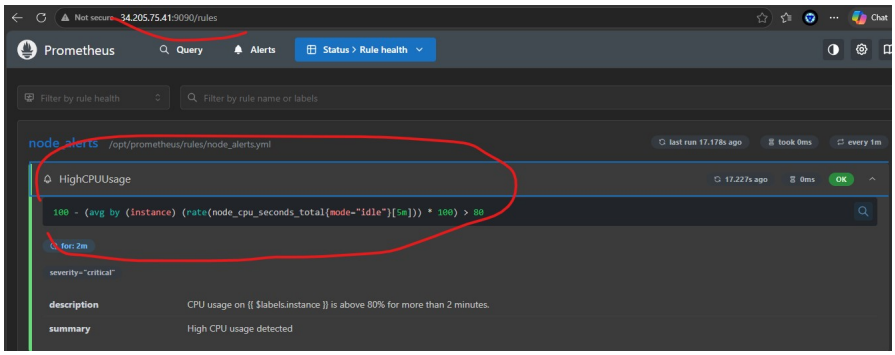
```
rule_files:
 - "/opt/prometheus/rules/*.yml"

scrape_configs:
 - job_name: "prometheus"
   static_configs:
    - targets: ["localhost:9090"]
 - job_name: "node-exporter"
   static_configs:
    - targets: ["10.0.0.204:9100"]
```



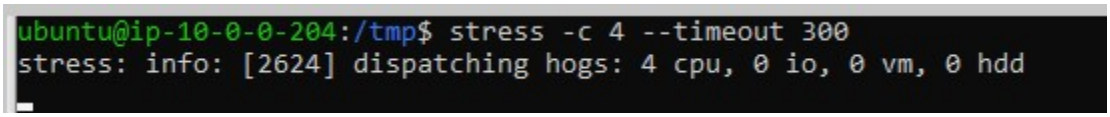
Test connection:

http://<EC2-1-PUBLIC-IP>:9090

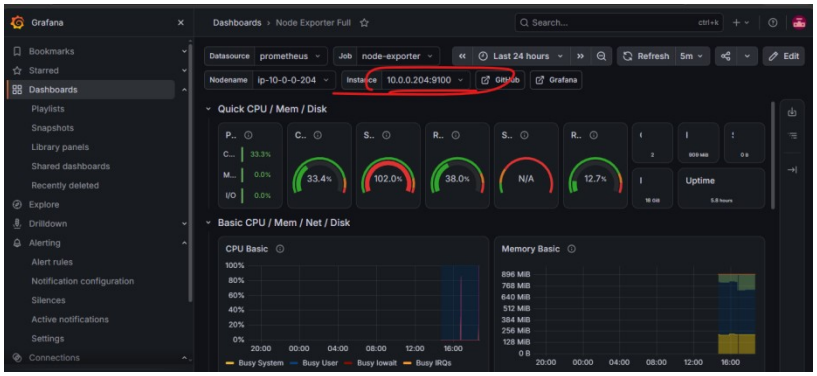
Mukesh Gouda 20/8/2026



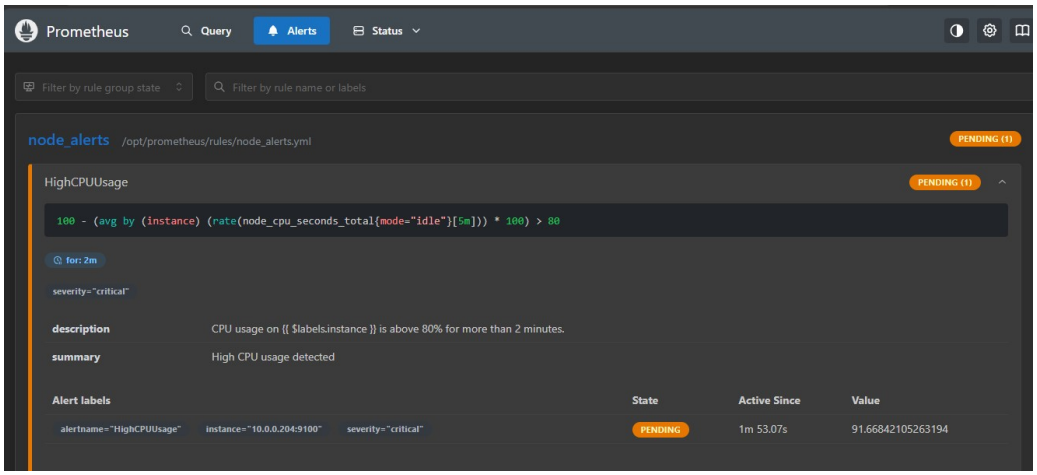
Now do the stressing using stress command in node-server(ec2-2).



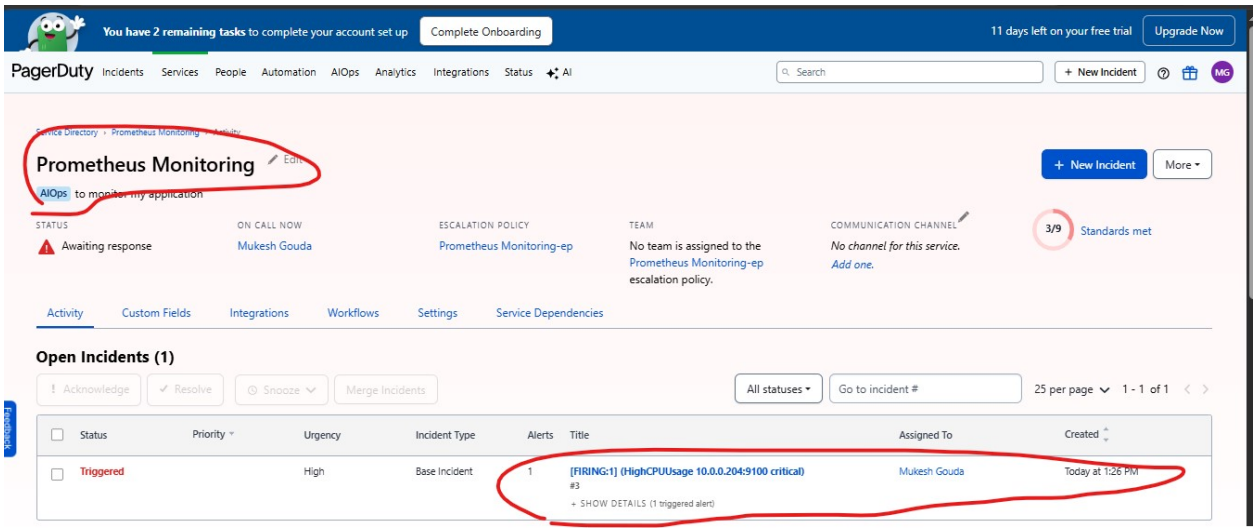
And observe in Grafana ,Prometheus and Pagerduty for incident.



Mukesh Gouda 20/8/2026



Wait till the alert in Prometheus in Firing state .



Dynamic monitoring issue :-

Dynamic Metric collection

Here we have manually added the node-server private ip to the yaml file.

But in dynamic situation like ASG it not possible to add manually and also remove once the server deleted .

To overcome this will need to implement the **EC2 service discovery** to auto identify the the server having node agent and tell the prometheous to scrape the data.

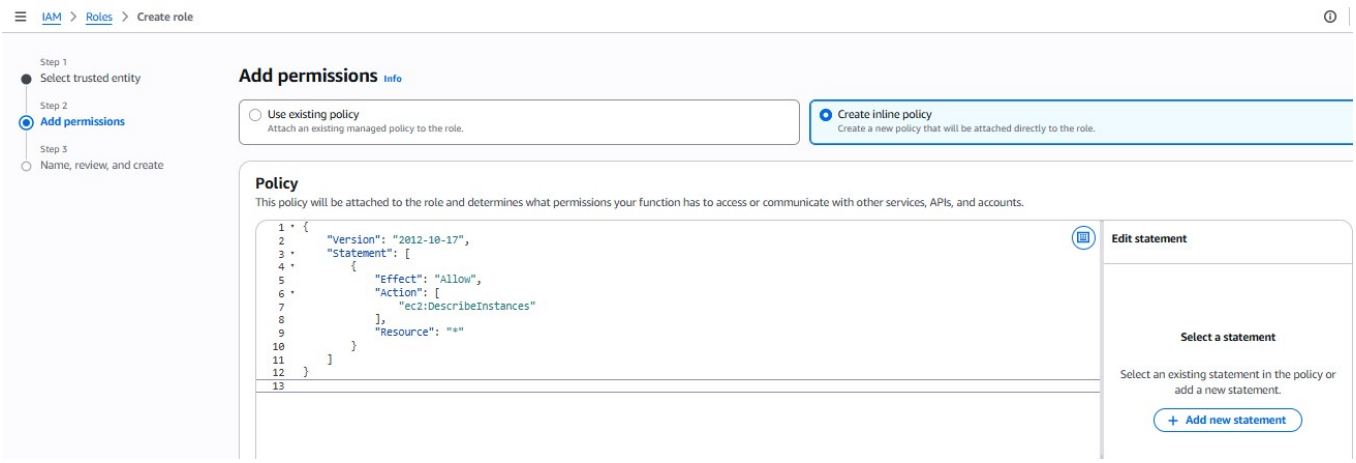
New EC2 launch → Node Exporter install → add tag (Monitoring=true) → Prometheus discovers it.

Mukesh Gouda 20/8/2026

Mandatory :

Create Prometheus EC2 IAM role → ec2:DescribeInstances

There wont be any existing policy we need to create an inline policy and attach it to Prometheus running server .




```
{
        "Version": "2012-10-17",
        "Statement": [
                {
                        "Effect": "Allow",
                        "Action": [
                                "ec2:DescribeInstances"
```

```
                ],
                "Resource": "*"
            }
        ]
}
```

When a new server created give naming tag

1 tag (Name:node-server)

2. open SG port :9100

3. configure node_exporter and make sure its running